Práctica 3.2 — Pipeline CI/CD con GitHub Actions

Objetivo: automatizar la construcción y publicación de la imagen del servidor del juego, y actualizar la flota con la etiqueta del commit.

Archivo: `.github/workflows/deploy.yml`.

Entregado en AWS (2026-11-09)

Se usó la variante AWS del material: `aws-actions/configure-aws-credentials`, `aws-actions/amazon-ecr-login@v2`, ECR y EKS.

Trabajo	Qué hace	Resultado
`pruebas`	pytest del servidor (82), del matchmaker (5) y de la tabla de líderes (20)	✅ 18 s
`smoke`	construye la imagen y juega una partida de 10 jugadores contra el contenedor; exige tick rate ≥ 19	✅ 22 s
`publicar`	sube `nebula-server:<sha>` y `matchmaker:<sha>` a ECR	✅ 18 s
`desplegar`	parcha la flota de Agones y el matchmaker en EKS con la imagen del commit, y espera servidores `Ready` con la versión nueva	✅ 24 s

Corrida: run 42694334223, commit `c295bf8`, 1 min 33 s en total. Los 3 servidores `Ready` quedaron con la imagen `nebula-server:c295bf8…`. Evidencia: `entrega/evidencias/3.2-pipeline/` (captura y salida de `kubectl`).

Seguridad: - **Identidad de GitHub:** usuario IAM `nebula-github` con permisos mínimos (subir a sus dos repositorios de ECR y `eks:DescribeCluster`). Dentro de Kubernetes solo puede parchar la flota y el matchmaker (`k8s/aws/rbac-github.yaml`). Su llave vive solo en los secretos de GitHub. - **Lo ideal habría sido OIDC, sin llaves guardadas** (estaba escrito en Terraform). La cuenta del plan gratuito lo bloquea: `iam:CreateOpenIDConnectProvider` tiene un *explicit deny* en la política de control de servicios de AWS. - **Imágenes inmutables:** ECR no deja reescribir una etiqueta, así que cada versión desplegada se rastrea a su commit. Además, ECR escanea vulnerabilidades al subir.

Versión original para Google Cloud

Paso	Estado
Código y Dockerfile de la 3.3	Hecho
Workflow escrito	Hecho. YAML validado; el trabajo `smoke` se ensayó en local con éxito (10 jugadores, tick rate 19.71, salida 0)
Repositorio en GitHub y pipeline en verde	Hecho: github.com/Vogelito09-lab/nebula-arena, primera corrida (2026-11-05) con `pruebas` y `smoke` en verde, `publicar` y `desplegar` omitidos (sin `GCP_PROJECT`)
Publicar en Artifact Registry y actualizar la flota	**Pendiente:** necesita la parte de nube de la 3.3

Etapas

```
push a main
  └─ pruebas    pytest: 14 unitarias/funcionamiento del servidor
      └─ smoke   docker build + partida de 10 jugadores contra el contenedor;
          │      falla si el tick rate baja de 19 (95 % de 20)
          └─ publicar   docker push .../nebula-server:<sha>        (si hay GCP_PROJECT)
              └─ desplegar  kubectl apply de fleet.yaml con <sha>;
                            espera a que haya un GameServer Ready con la versión nueva
```

- **Si una etapa falla, las siguientes no corren.** Una prueba roja nunca publica una imagen.
- En un *pull request* corren `pruebas` y `smoke`; publicar y desplegar solo ocurren con *push* a `main`.
- Mientras el repositorio no tenga la variable `GCP_PROJECT`, `publicar` y `desplegar` aparecen como omitidas (*skipped*) y el pipeline queda en verde. Al configurarla se activan solas.

Configuración (cuando exista el proyecto de Google Cloud)

1. Crear la cuenta de servicio. Terraform (3.6) la crea como `nebula-dev-github` con los roles `artifactregistry.writer` y `container.developer`. Sin Terraform, con los comandos del material:

```
bash gcloud iam service-accounts create github-deployer gcloud projects add-iam-policy-binding <PROYECTO> \ --member=serviceAccount:github-deployer@<PROYECTO>.iam.gserviceaccount.com \ --role=roles/artifactregistry.writer gcloud projects add-iam-policy-binding <PROYECTO> \ --member=serviceAccount:github-deployer@<PROYECTO>.iam.gserviceaccount.com \ --role=roles/container.developer # para actualizar la flota gcloud iam service-accounts keys create key.json \ --iam-account=github-deployer@<PROYECTO>.iam.gserviceaccount.com
```

2. En GitHub, *Settings → Secrets and variables → Actions*: - **Secret** `GCP_SA_KEY`: el contenido de `key.json`. Después **borra** `key.json` de tu PC. `.gitignore` ya lo excluye, pero no debe quedar en ningún lado. - **Variable** `GCP_PROJECT`: el ID del proyecto.

3. El clúster debe llamarse `nebula` en `us-central1-a` (el de la 3.3). Si lo creaste con Terraform se llama `nebula-dev`: cambia `CLUSTER` al inicio del workflow.

Drenado: por qué no se cortan partidas

`kubectl apply` de una flota con imagen nueva hace que Agones cree servidores nuevos y borre **solo los que no tienen partida** (`Ready`). Los `Allocated` siguen con la versión anterior hasta que su partida termina y llaman a `/shutdown`. Lo comprobamos en local: en el paso 9 de `evidencias/3.3-agones-local.txt`, la flota pasó de 1.0 a 1.1 reemplazando solo los servidores libres.

Rollback (párrafo del entregable)

Cada versión es una imagen inmutable etiquetada con el SHA de su commit, así que volver atrás es apuntar la flota a la etiqueta anterior. Hay dos formas:

1. **Revertir el commit** (`git revert <sha>` y *push*). El pipeline vuelve a probar, construye la imagen del código anterior y despliega. Queda registro en el historial.
2. **Rollback inmediato** sin esperar al pipeline, con la imagen anterior que ya está en Artifact Registry:

`bash gcloud artifacts docker images list us-central1-docker.pkg.dev/<PROYECTO>/nebula/nebula-server # ver etiquetas sed -i "s|nebula-server:.*|nebula-server:<SHA-ANTERIOR>|" k8s/fleet.yaml kubectl apply -f k8s/fleet.yaml`

En ambos casos aplica el mismo drenado: las partidas en curso con la versión mala terminan normalmente, y solo las partidas nuevas usan la versión anterior.

Entregable

- [ ] Captura del pipeline en verde (después de subir el repositorio a GitHub)
- [ ] Imagen publicada con la etiqueta del commit (después de la parte de nube de la 3.3)
- [x] Párrafo de rollback (arriba)